

SYSTEM REFACTOR SPECIFICATION

Smart Wealth Tracker - Cloudflare Free Edition

Objective

You are a Senior Software Architect and Senior Full Stack Engineer.

Your task is to refactor and upgrade the existing Smart Wealth Tracker project into a long-term production-ready application while keeping compatibility with all existing data.

This project is intended for long-term family usage (maximum 4 users) and **MUST** continue running on Cloudflare Free Plan.

The goal is reliability, security, maintainability and scalability.

Do NOT rewrite the project from scratch.

Instead:

- Analyze the existing project.
- Preserve existing functionality.
- Refactor architecture where necessary.
- Keep UI/UX familiar.
- Improve code quality.
- Improve security.
- Improve storage architecture.
- Preserve all existing data.

IMPORTANT REQUIREMENTS

This project already contains production data.

NO DATA LOSS IS ACCEPTABLE.

Migration must be safe.

Existing backup files **MUST** remain usable forever.

TARGET ARCHITECTURE

Cloudflare Pages

↓

Cloudflare Workers API

↓

Cloudflare D1 (Main Database)

↓

Cloudflare R2 (Image Storage)

↓

Cloudflare KV (Session / Cache / Config Only)

↓

Cron Trigger

↓

Google Drive Backup

Everything must remain compatible with Cloudflare Free Plan.

Do not introduce paid services.

PHASE 1

Analyze Existing Project

Before changing anything:

Analyze

- Existing architecture
- Existing storage
- Existing login
- Existing backup
- Existing APIs
- Existing upload system

Generate a migration plan before modifying any code.

PHASE 2

Database Migration

Current storage should no longer be the primary database.

Create Cloudflare D1 database.

Design proper normalized schema.

Suggested tables:

users

accounts

transactions

categories

attachments

audit_logs

settings

sessions

backup_history

Every table must include

CreatedAt

UpdatedAt

DeletedAt (Soft Delete)

DATA MIGRATION

Create migration scripts.

The migration must support

Current Storage

↓

Cloudflare D1

WITHOUT losing data.

BACKWARD COMPATIBILITY

Current Backup JSON MUST still be importable.

Never break compatibility.

Implement Backup Versioning.

Example

Version 1

Current Backup

↓

Import

↓

Auto Convert

↓

Store into latest schema

Future backup versions must also remain compatible.

IMPORT SYSTEM

Create Backup Import Wizard.

Steps

1. Select Backup File

2. Analyze File

3. Show Preview

Accounts

Transactions

Categories

Attachments

Backup Date

Backup Version

4. Validation

5. Import

Support

Replace

Merge

Skip Duplicate

Never overwrite data without confirmation.

EXPORT SYSTEM

Daily Backup should generate

ZIP File

Containing

database.json

metadata.json

manifest.json

attachments

Metadata should contain

Backup Version

Application Version

Database Version

Backup Time

Record Counts

Checksum

Created By

AUTOMATIC BACKUP

Every day

00:00

Cloudflare Cron Trigger

Automatically

Export Database

Compress

Upload to Google Drive

Backup Folder configurable.

Retention

30 days

Delete backups older than 30 days.

Do NOT overwrite today's backup.

Always verify uploaded file integrity.

If backup fails

Notify immediately.

IMAGE STORAGE

Move image storage to Cloudflare R2.

Before upload

Resize

Maximum width

1600px

Convert

WebP

Quality

80

Automatically reduce image size.

Store only optimized images.

Keep original filename only as metadata.

Store randomized filename using UUID.

SECURITY

Upgrade entire authentication system.

Current system uses password only.

Replace with

Username

Password

Role

Admin

User

Only Admin can

Create Users

Delete Users

Reset Password

View Audit Logs

Manage Settings

PASSWORD

Never store plain passwords.

Use Argon2id.

If unavailable

Use bcrypt.

SESSION

Use

Secure Cookie

HttpOnly

SameSite=Lax

Never use LocalStorage for authentication.

RATE LIMIT

Protect login endpoint.

Example

5 failed attempts

↓

Lock account

15 minutes

SECURITY HEADERS

Implement

CSP

HSTS

X-Frame-Options

X-Content-Type-Options

Referrer Policy

AUDIT LOG

Record every

Login

Logout

Insert

Update

Delete

Restore

Backup

Settings Change

Include

User

Timestamp

IP

Action

Previous Data

New Data

SOFT DELETE

Never permanently delete records.

Use Soft Delete.

Provide Trash Page.

Allow Restore.

DASHBOARD

Admin Dashboard

Display

Database Status

Cloudflare D1 Status

Cloudflare R2 Status

Cloudflare KV Status

Cron Status

Google Drive Status

Last Backup

Storage Usage

Database Size

Image Size

Total Users

Total Accounts

Total Transactions

Application Version

DAILY REPORT

Every day

20:00

Automatically send report.

Preferred

LINE Notify replacement using LINE Messaging API.

If another free notification method is more appropriate, implement the best alternative.

Include

Total Balance

Balance of every account

Today's Income

Today's Expense

Future Expenses

Monthly Income

Monthly Expense

Upcoming Due Payments

Backup Status

Storage Status

Database Health

Any Error

SEARCH

Improve search performance.

Support

Date Range

Category

Account

Keyword

Amount

Income

Expense

PERFORMANCE

Optimize

API

Database Query

Image Loading

Pagination

Caching

Avoid unnecessary reads.

CODE QUALITY

Refactor entire project.

Remove duplicated logic.

Split into modules.

Use clean architecture.

Meaningful naming.

Strong typing.

Consistent style.

ERROR HANDLING

Never silently fail.

Every exception should

Log

Display friendly message

Write audit log

TESTING

Before finishing

Test

Login

CRUD

Image Upload

Backup

Restore

Migration

Import Old Backup

Export

Daily Report

Permission

Admin

User

MIGRATION ORDER

Never replace everything at once.

Perform

1.

Analyze

2.

Backup Existing Data

3.

Create D1 Schema

4.

Migration Script

5.

Import Existing Data

6.

Verify

7.

Switch System

8.

Regression Test

9.

Remove Legacy Storage

BACKUP COMPATIBILITY

The project already has backup files in the old JSON format.

Those backup files MUST continue to work.

Implement automatic conversion if needed.

Never require users to manually modify backup files.

CLOUDFLARE FREE PLAN

Everything must remain compatible with Cloudflare Free.

Prefer

Cloudflare Pages

Cloudflare Workers

Cloudflare D1

Cloudflare R2

Cloudflare KV

Cloudflare Cron Trigger

Do not introduce unnecessary paid services.

FINAL DELIVERABLE

Do not stop after partial implementation.

Complete every phase.

At the end provide

Architecture Diagram

Database Schema

Migration Report

Security Report

Testing Report

Remaining TODO (if any)

Known Limitations

Future Suggestions

Only finish when the project is production-ready and all existing data remains compatible.